

# Agentic AI

## Mastering Graph RAG: Bridging Semantic Search and Structured Knowledge

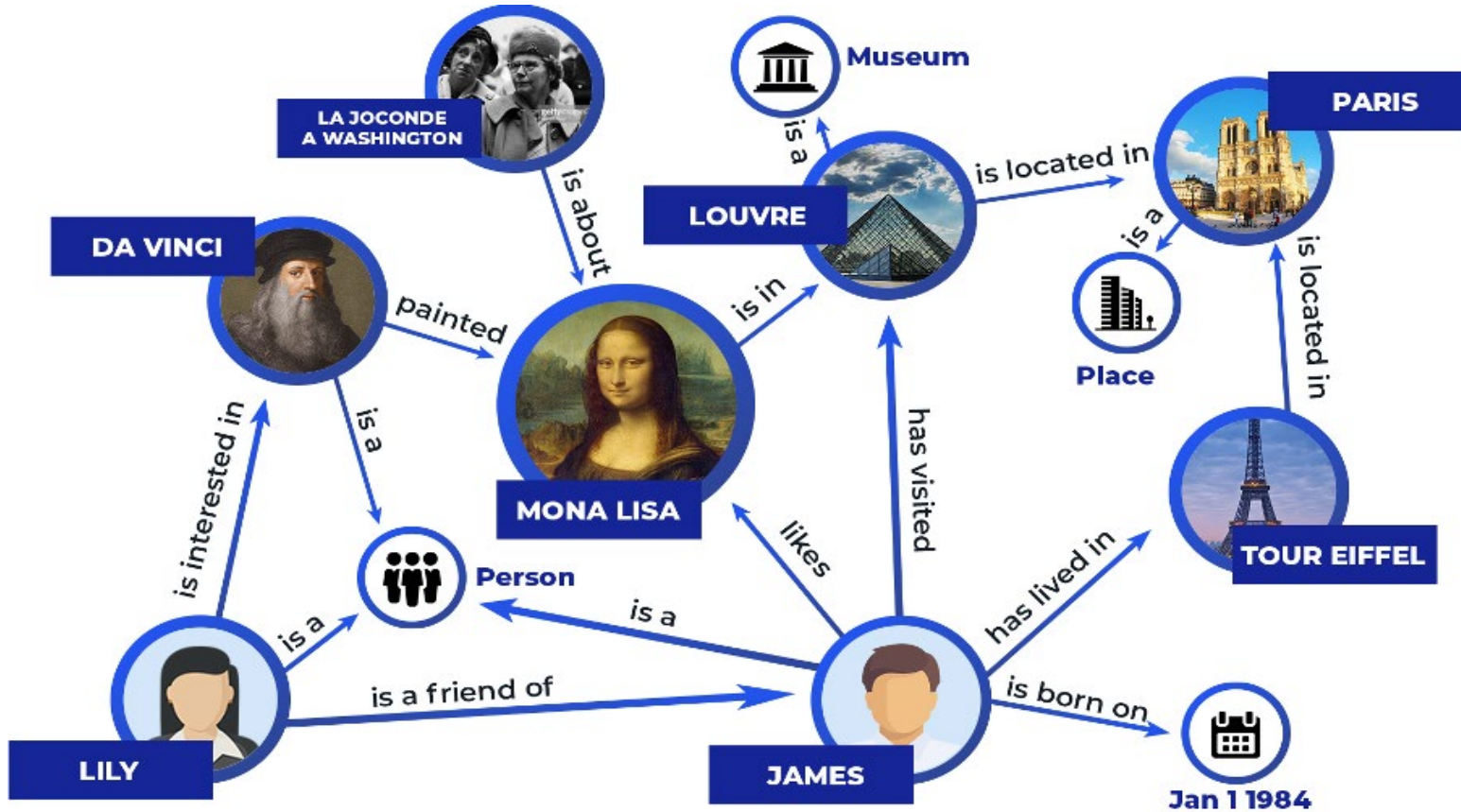
# Introduction to Knowledge Graphs

## Core Definition

- A knowledge graph is a network of real-world entities—people, places, concepts, or things represented as nodes and linked by explicitly defined edges (relationships). Formally, it is a directed property graph  $G = (V, E)$ :
- **Vertices (V):** Possess labels and internal property key-value pairs, e.g., `(:Project {name: "Wastewater Surveillance", status: "Active"})`
- **Edges (E):** Typed, directed connections, e.g., `[ :UTILIZES ]` or `[ :MANAGED_BY ]`

**i** Semantic meaning is explicitly embedded within the structural topology itself. These relationships are not latent inside text or model weights.

# Knowledge Graph Example



Source: [What Are Knowledge Graphs? A Comprehensive Guide to Connected Data](#)

# The Knowledge Graph Landscape

## Enterprise Domain Applications

---

### Web Search

Google, Yahoo, Bing use entity mapping to power semantic search queries, rich infoboxes, and intent discovery.

### Social Networks

Facebook, Instagram, LinkedIn, X map explicit user connection topologies, behavioral traits, and corporate networks.

### Commerce

Amazon, Airbnb, eBay, Uber optimize logistics, recommendation algorithms, pricing graphs, and user-to-product mapping.

### Finance and Strategy

Bloomberg, Accenture build cross-company dependency indices, market risk systems, and supply-chain intelligence layers.

## Open and Collaborative Foundations

---

### Wikidata

A collaboratively edited, multilingual semantic core hosted by the Wikimedia Foundation, serving as a global standard for deterministic entity identifiers.

### DBpedia and YAGO

Academic frameworks that turn unstructured raw wiki data into linked networks, cleanly merging crowdsourced breadth with rigid linguistic taxonomies like WordNet.

# Traditional vs. Graph RAG

- **Traditional (Vector-Only) RAG:**

- **Mechanism:** Top-K similarity matching over vector embeddings of text chunks.
- **Failure Modes:**
  - **Global Aggregation:** Struggles with global aggregation queries and fails when critical context is split across arbitrary chunk boundaries.
  - **Suffers from the Granularity Dilemma:** vector-only retrieval excels at fuzzy semantic matching but fundamentally fails at multi-hop relational reasoning and global aggregation queries.

- **Graph-Augmented RAG:**

- **Mechanism:** Merges text embeddings with explicit entity-relationship topologies.
- **Superpowers:** Maintains global thematic summaries, and preserves structural relationships across a disparate corpus.

# Why Graph RAG is Critical for LLMS

## Beyond Semantic Similarity

- 🔗 **Multi-Hop Reasoning** Traverses physical connections to answer complex questions that span multiple disconnected document silos.
- 🛡️ **Factual Grounding** Replaces probabilistic "next-token" guesses with deterministic symbolic links to verified entities.
- ⚡ **Context Efficiency** Feeds only relevant sub-graphs to the LLM, preventing "lost in the middle" data saturation.
- 🔍 **Explainability** Provides a traceable path from the generated answer back to the original source records.
- 🌐 **Macro-Level Corpus Comprehension** Maps the entire document landscape into communities using network clustering.



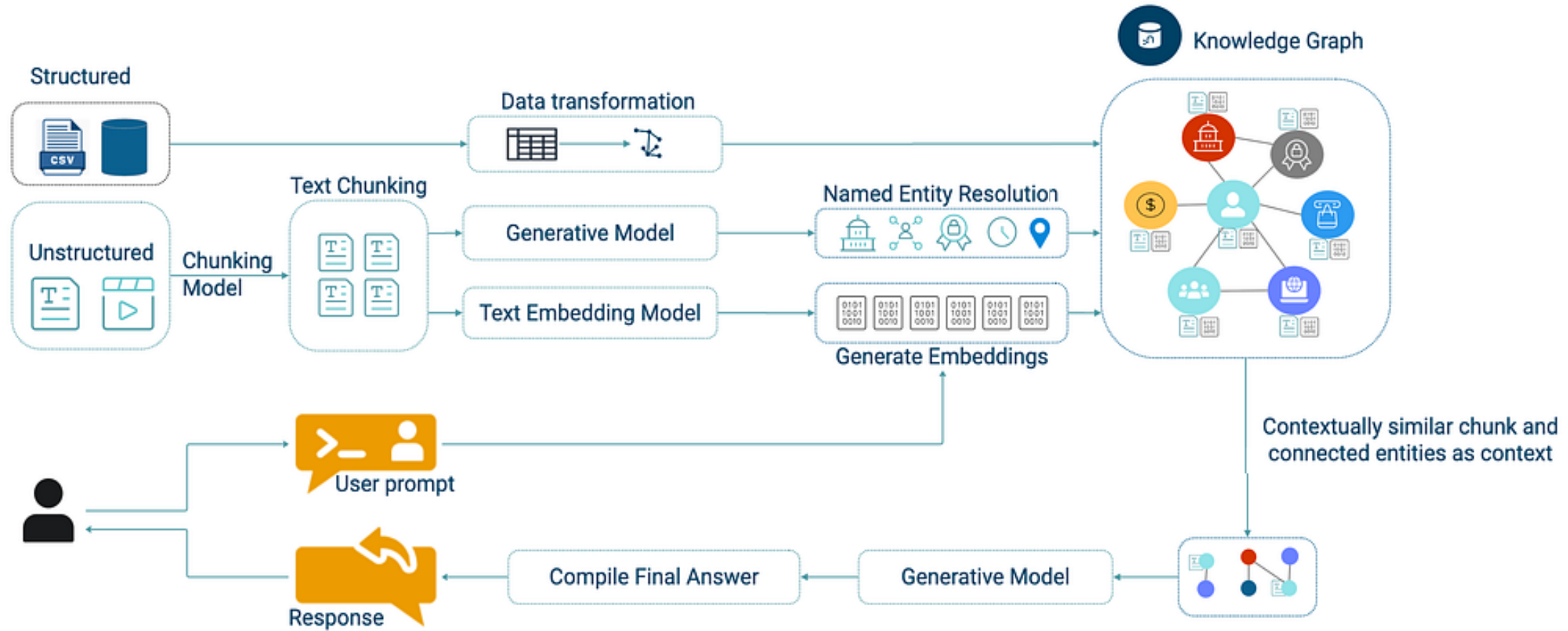
# Architectural Tradeoffs

| Dimension       | Traditional Vector RAG                                                       | Graph RAG Systems                                                                |
|-----------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Ingestion Cost  | <b>Low</b> —linear $O(N)$ chunking and embedding                             | <b>High</b> —non-linear LLM/NLP extraction pipeline                              |
| Retrieval Speed | <b>Fast</b> —optimized ANN (approximate nearest neighbor) search             | <b>Variable</b> —depends on traversal depth and sub-graph size                   |
| Reasoning Depth | <b>Surface-level</b> — limited to local semantic overlap within a text block | <b>Deep</b> —traverses explicit physical paths across multiple documents         |
| Query Type      | <b>Specific/Local</b> — <i>"What is the deadline for project X?"</i>         | <b>Global/Relational</b> — <i>"How are all 2026 initiatives interconnected?"</i> |

❏ Modern Graph RAG does not *replace* vectors—it **unifies** them with explicit graph topology into a single hybrid retrieval engine.

# Graph RAG in Practice

# Architecture of Graph RAG Systems



Source: GraphRAG and Agentic Architecture

# Creating a Graph-based Index

## 1. Text Segmentation and Chunking

Unstructured text is parsed using token-aware sliding window splitters (e.g., 600-token windows with 100-token overlap) to preserve paragraph-level semantic integrity across chunk boundaries.

## 2. LLM-Driven Triplet Extraction

An LLM extracts atomic subject-predicate-object facts from each chunk via a strict deterministic schema prompt:

$$\text{Text Chunk} \longrightarrow (\text{Subject}) \xrightarrow{[\text{PREDICATE}]} (\text{Object})$$

## 3. Entity Resolution and De-duplication

Synonymous entities (e.g., *"United States of America"* and *"USA"*) are merged into unified canonical nodes using entity linking and clustering algorithms, eliminating structural ambiguity.

## 4. Graph Storage and Lineage Linking

Structured entities and attributes are written to the property graph. New structural nodes are simultaneously linked back to source chunk tracking IDs, maintaining complete data lineage.

**Graph construction is a deterministic, four-stage pipeline that transforms raw unstructured text into a richly structured, query-ready property graph.**

# Building Hierarchical Community

- Hierarchical clustering methods partition a graph into dense, isolated sub-graphs by maximizing internal community connection density relative to random inter-community edges.

## Granular Level

Small, tightly focused clusters representing specific entities and their immediate context—e.g., a localized project sub-team and its direct deliverables.

## Intermediate Level

Broader organizational clusters representing business tracks, domain-level programs, or cross-functional initiatives spanning multiple granular clusters.

## Macro-Global Level

Top-level summaries covering entire corporate initiatives or thematic domains. Pre-computed by LLM, vector-indexed, and cached directly within the graph topology.

① The **Louvain & Leiden** methods are popular unsupervised clustering algorithms

# The Prompt/Query Workflow

## 1. Input Vectorization

Raw query text is captured and encoded into a multi-dimensional dense embedding vector by the orchestration layer.

## 2. Hybrid Query Routing

System selects: localized graph traversal, abstract global community scan, or raw vector chunk search—based on query classification.

## 3. Graph Traversal

Query engine traces paths, verifies edge conditions, and filters node properties across relevant sub-graphs using structural constraints.

## 4. Context Aggregation

Matches are flattened to text tables, sorted by relevance metrics, and injected into the target context window for prompt construction.

## 5. Constrained Generation

The combined prompt forces the LLM to answer strictly from the injected evidence graph, guaranteeing factual auditability.

# Retrieval Strategies: Local Search

## Micro-Targeted Graph Traversal

- **Objective:** Answer explicit, entity-focused questions about specific nodes and their immediate interactions—e.g., "Which vendor supplies component X to our manufacturing plant?"
  1. **Semantic Seed Search:** Vector lookup against the entity node index to identify core anchor nodes.
  2. **Bounded Horizon Expansion:** Fixed N-hop traversal from seed anchors to collect adjacent neighbors.
  3. **Context Enrichment:** Assembles node property dictionaries, edge descriptions, and source text block references.

□ Highly efficient, low context overhead, precise factual retrieval.

# Retrieval Strategies: Global Search

## Holistic Dataset Comprehension

- **Objective:** Resolve broad, cross-corpus exploratory questions—e.g., "What are the primary systemic risks across all audit records?"
  1. **Community Selection:** Query vector matched against pre-computed hierarchical community summaries.
  2. **Map Stage:** Query distributed across parallel worker LLM instances, each generating a partial report from its community block.
  3. **Reduce Stage:** Intermediate responses merged, duplicates filtered, and synthesized into a single cohesive global summary.

❏ Higher token cost due to parallel LLM calls—essential for disconnected, large-scale corpora.

# The Graph RAG Technology Stack



## Neo4j

Industry-standard native property graph platform with graph-native storage layouts, HNSW vector indexing, and built-in Graph Data Science (GDS) runtime for advanced analytics.



## Microsoft GraphRAG

Open-source data pipeline engineered explicitly around LLM-based entity extraction, Leiden community clustering, and modular global/local search architectures.



## Ontotext GraphDB/Amazon Neptune

RDF triple stores optimized for W3C standards compliance, SPARQL query execution, and formal ontology definitions—ideal for regulated enterprise environments.



## LangChain/LlamaIndex

Production-ready orchestration frameworks providing memory routers, graph storage wrappers, and unified pipeline integrations for vector-graph hybrid querying.



## Neo4j Bloom/Gephi

Visual graph exploration tools—Bloom for NLP-driven business user interaction, Gephi for open-source statistical layout and analysis of massive-scale network topologies.

# Tech Stack Tradeoffs

| Evaluation Vector     | Enterprise Native (e.g., Neo4j)                                               | Open-Source/Semantic RDF                                           |
|-----------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Data Layout Paradigm  | Property Graph: Highly expressive key-value pairs on nodes/edges.             | Triple Store: Rigid Subject-Predicate-Object atomic facts.         |
| Query Runtimes        | Optimized for deep, localized, multi-hop path traversals.                     | Optimized for globally standardized semantic web pattern matching. |
| Schema Flexibility    | Schema-lite/Optional: Highly adaptive for rapid LLM entity extractions.       | Schema-strict: Strongly typed, predefined ontology structures.     |
| Ecosystem Integration | Deep native integrations with Graph RAG orchestrators (LangChain/LlamaIndex). | Requires custom mapping layers to bind text text embeddings.       |

# The Graph RAG Paradigm Shift



## The Hybrid Stack

Unify semantic vectors with symbolic property graphs for atomic, multi-hop precision and verifiable evidence chains.



## Scalable Comprehension

Leverage hierarchical community summaries to achieve global data synthesis and macro-level corpus understanding.



## Rigid Governance

Secure autonomous retrieval with explicit policy constraints, identity verification, and deterministic audit trails.

**Graph RAG allows us to transform statistical generation into deterministic intelligence.**

